Deep Reinforcement Learning for Analog Circuit Sizing

Zhenxin Zhao and Lihong Zhang
Department of Electrical and Computer Engineering
Faculty of Engineering and Applied Science
Memorial University of Newfoundland, St. John's, Canada

***Abstract*—Automated analog circuit sizing is always a challenging task, due to high complexity involved, huge design space searched, and conflicting constraints traded off. This paper proposes an automated trial and error approach that combines reinforcement learning with deep learning for analog circuit sizing. Through the self-improvement learning way, the proposed method behaves like a designer, who learns from trials and derives experience, evolving itself to finally discover the sizes that satisfy the performance specification based on simulation results. In order to greatly reduce the number of simulations, we propose a symbolic filter that builds a polynomial equation system by utilizing the curve-fitting results and then applies the worked out small-signal parameter values to implement symbolic analysis to quickly evaluate the circuit performance, passing only the satisfied ones to the simulator. Our experimental results demonstrate the reliability of the proposed method, and also reveal the self-improvement capability.**

Keywords—analog circuit sizing; deep reinforcement learning; curve-fitting; polynomial equation system; symbolic analysis

I. INTRODUCTION

Manual analog circuit design, which is an intensive and time-consuming task even for an experienced and highly skilled engineer, has become increasingly difficult due to the continuous scaling of IC technologies and the increasing demands for high-performance, low-power designs [1]. By greatly facilitating human laborious work, analog electronic design automation (EDA) tools can not only significantly improve the design productivity for reduced time-to-market, but also enhance the design accuracy. For the analog circuit design procedure before physical design [2], circuit synthesis, which includes circuit topology synthesis and sizing, have attracted a lot of research interests. Compared with schematic-level topology synthesis [3], sizing is more complicated since it usually needs to consider layout parasitics [4][5].

Automated analog circuit sizing approaches can be classified into two categories, the model-based and simulation-based [6]. Model-based methods use simplified equations or polynomials to model the circuit performances, then solve the optimization problem based on the calculated performance results of these models. However, the predicted performances from the circuit models are not real circuit performances, which makes the mode-based methods cannot guarantee the accuracy for the large-scale circuits. In contrast, simulation-based methods directly rely on the simulation results to drive the optimization process, therefore they are generally more reliable and accurate.

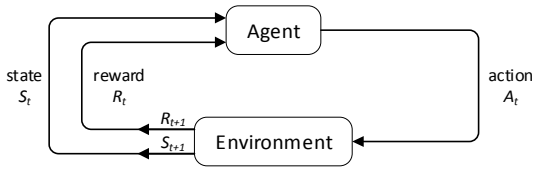


Fig. 1. The agent-environment interaction in reinforcement learning loop.

However, compared with model calculation, computation-intensive simulation is time-consuming, especially for large-scale circuits, leading to the simulation-based methods cost large time on simulation during the optimization process.

Our proposed deep-reinforcement-learning-based approach is essentially a simulation-based approach, which relies on the simulation results to assign reasonable rewards directing the learning process. Deep neural network (DNN) is employed in this work due to its strong scalability, which is naturally suitable for the complex sizing problem. Through successive trials, the algorithm itself is continuously learning from the rewards and improving its way to find the optimum solution by figuring out which parameter should change to which amount. This self-improvement trial and error method can reliably discover solutions, but on the other hand, the runtime cost is expensive due to a large number of simulations. To address this issue, in this paper, we propose a symbolic filter that can fast evaluate the performance via symbolic analysis, which quickly filters the trials that do not meet the targets, thus greatly reducing the number of time-consuming simulations.

The rest of this paper is organized as follows. Section II discusses the details about our proposed analog circuit sizing framework. Section III explains the components in the proposed reinforcement learning system. Section IV reports our experimental results and Section V concludes the paper.

II. THE ANALOG CIRCUIT SIZING FRAMEWORK

A. Preliminaries

Reinforcement learning (RL) is learning from (positive and negative) rewards. When an agent is at a concrete state ($s \in S$) and takes a valid action ($a \in S$), an immediate reward ($r \in R$) is obtained from the environment, where the capital letter and the lower-case letter denote a set and a specific instance of the set, respectively [7].

The interaction between agent and environment is depicted in Fig. 1. Specifically, agent is a function that transforms the

978-1-7281-3320-1/20/$31.00 ©2020 IEEE
Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 18:05:54 UTC from IEEE Xplore. Restrictions apply.

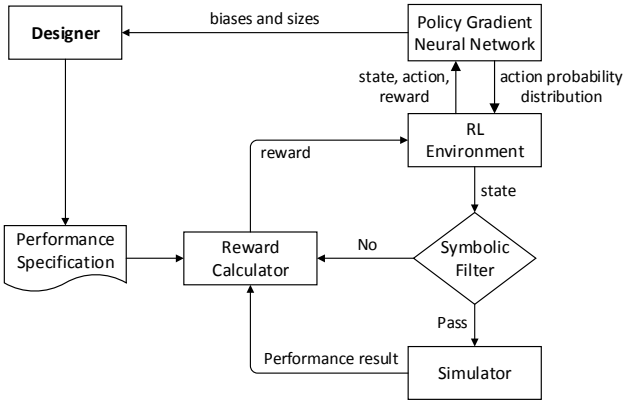


Fig. 2. The framework of our proposed deep-reinforcement-learning-based analog circuit sizing.

current (time step t) state and reward into an action; environment is a function that converts an action taken in the current (time step t) state into the next (time step t+1) state and reward. This RL loop outputs a sequence of states, actions and rewards. The objective of the agent is to learn sequences of actions that will maximize its expected cumulative rewards. The expected cumulative reward at time step t can be written as:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \tag{1}$$

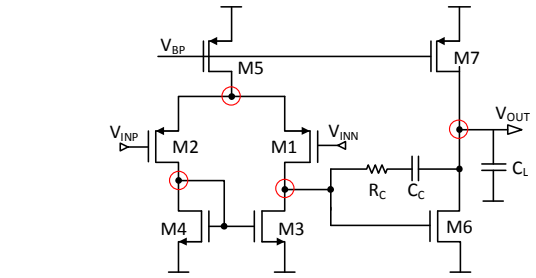
where $\gamma \in (0, 1]$ is called the *discount factor*, which means future rewards should worth less than immediate rewards due to the reason that rewards coming sooner are more probable to happen.

B. Deep Reinforcement Learning System for Sizing

In this paper, the circuit sizing process is modeled as training a policy gradient based RL system, as depicted in Fig. 2. Policy (π) is the strategy that an agent employs to determine the next action based on the current state, which maps the state to the probability distribution over actions. In this system, a supervised classification model, policy gradient neural network (PGNN), is built to predict the changes of circuit parameter values, which outputs the probability distribution over all valid actions (a) under the input state (s)

$$\pi_\theta(s) = P[a|s]. \tag{2}$$

As depicted in Fig. 2, the RL environment provides the current training data tuple (state, action, and reward) to the PGNN and gets the action probability distribution under the current state in turn. Considering the trade-off between exploitation and exploration to overcome the risk of getting stuck in a sub-optimal solution, the action to be taken can be inferred (e.g., ε-greedy method). By executing the action, the agent gets the reward and the next state from the environment, which forms a new training data tuple to train the PGNN model. This RL loop continues and the gradient ascent method is employed to iteratively find the best weights that improve the policy in the direction of maximizing the expected future reward, with the update rule:



(a)

V_{INP}	V_{BP}	L_1	W_1	L_3	W_3	L_5	W_5	L_6	W_6	L_7	W_7	R_C	C_C

(b)

Fig. 3. (a) Two-stage OpAmp. (b) Its state representation.

$$\Delta\theta = \alpha * \nabla_\theta\big(log_\pi(s, a, \theta)\big) * r(\tau) \tag{3}$$

where α is the learning rate and $r(\tau)$ is the expected future reward.

In our work, the state encodes the values of circuit parameters, which will be explained in detail in Section III. The evaluation of the circuit's performance with those parameter values encoded in the state is carried by two means, our proposed symbolic filter and the simulator. The symbolic filter quickly but roughly evaluates some performance attributes through symbolic analysis, while the simulator accurately but slowly assesses all the performance attributes via SPICE-simulation. As depicted in Fig. 2, only the trials that pass the symbolic filter would be further assessed by the simulator; otherwise, rewards would be assigned zeros and returned, then a new trial would be invoked. For the trials that successfully passed to the simulator, rewards would be generated based on their performance results and the target specifications, which will be illustrated in detail in Section III.

In an RL system, the training and the consulting of policy can be done simultaneously, but usually, training is done to some extent before using the policy. Different from the training of policy, which needs to consider the trade-off between exploitation and exploration, the using of policy only chooses that actions that have the highest probabilities. In this way, the users of PGNN can get the best result that the RL agent learned so far.

III. Components in The Proposed RL System

A. State and Action

Circuit parameters, including biases, sizes, resistances, and capacitances, can be classified into fixed parameters (e.g., the power voltage) and changeable parameters under certain CMOS technology. In the proposed method, we encode the values of changeable circuit parameters into the state, which is in the representation of an array. For the symmetric transistors, only one transistor's parameter values are encoded. An example circuit, two-stage OpAmp, is given in Fig. 3(a). Considering the symmetries among transistors, 14 changeable parameters' values are encoded in its state, which is presented in Fig. 3(b).

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 18:05:54 UTC from IEEE Xplore. Restrictions apply.

The reason for representing the state in this way lies in that those parameter values can be directly fed to simulation.

In our scenario, action is defined as changing the value of a parameter in the state. For each parameter, there are two actions to make a change, increment and decrement. The amount that an action going to change is depended on the product of two user-defined hyperparameters, *change rate* and *change capacity*. *Change rate* is uniform, while for distinct types of parameters, different values can be assigned to the *change capacity*.

B. Action Validity

Circuit constraints can be effectively handled by defining the validity of certain actions. For the example of two-stage OpAmp shown in Fig. 3(a), it imposes the relationships that V_{BP} must larger than V_{INP} but both V_{BP} and V_{INP} must be smaller than V_{DD}. If executing an action leads to break these constraints, this action is invalid. By permitting only the valid actions to be executed, unreasonable combinations of parameter values are largely eliminated, resulting in significantly reducing the number of simulations and improving the efficiency of the sizing process.

C. Reward

In the proposed RL system, the design objectives are encoded into the rewards to instruct the learning direction. Even though reinforcement learning is single-objective based, multi-objective optimization still can be effectively handled by our defined reward function, which can be formulated as:

$$r(t) = \begin{cases} \sum k_i * \frac{PC_i}{PS_i}, & if\ PC_i > PS_i\ \ for \forall\ i \\ \sum k_i, & elif\ PC_i > PS_i\ for \exists\ i \\ 0, & else \end{cases} \quad (4)$$

where *PC* and *PS* refer to the simulation calculation results and the given performance specifications, respectively. Each performance attribute *i* has a coefficient k_i to weight itself, which is set by designers. Positive $k_i \in (0, 1)$ aims to maximize a performance attribute (e.g., dc gain), while negative $k_i \in (-1, 0)$ intends to minimize a performance attribute (e.g., power consumption).

D. Symbolic Filter

The proposed symbolic filter aims to apply the symbolic analysis method to fast evaluate partial performance attributes (i.e., the dc gain and phase margin in this work). However, as depicted in Fig. 2, the input to the symbolic filter is just the values of sizes and biases that encoded in the state, while to implement symbolic analysis, the values of small-signal model parameters (e.g., g_m, g_{ds}, C_{gd}, C_{gs}) for all transistors should be known. Therefore, as shown in Fig. 4, there exists a parameter gap between them. This contribution addresses the gap through the building and solving of a polynomial equation system, which relies on the curve-fitting results.

The curve-fitting technique is utilized to derive the transistor models that reflect the parameter mappings ① and ② as depicted in Fig. 4. By applying the transistor modeling

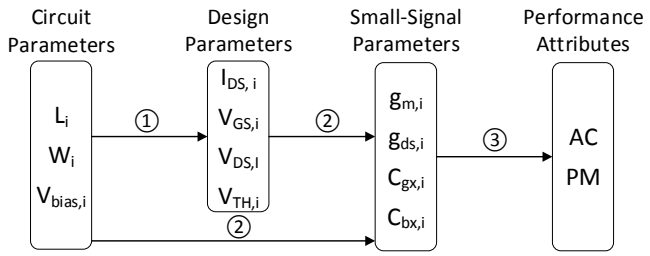


Fig. 4. Parameter mappings in our proposed symbolic filter.

method developed by [8], parameter mappings ① and ② can be settled for different segments of channel length for NMOS and PMOS transistors, respectively. Here, we list two of NMOS transistor as examples:

$$I_{ds,i} = a_1 W_i {L_i}^{a_2} (V_{GS,i} - V_{th,i})^{a_3} {V_{DS}}^{a_4} \quad (5)$$

$$g_{m,i} = b_1 W_i {L_i}^{b_2} (V_{GS,i} - V_{th,i})^{b_3} {V_{DS}}^{b_4} \quad (6)$$

where *i* refers to the i_{th} transistor in the circuit, and a_1, a_2, a_3, b_1, b_2, b_3 are coefficients.

The issue is that some transistors' V_{GS} and V_{DS} are unknown, leading to the values of their small-signal model parameters cannot be calculated. For the two-stage OpAmp depicted in Fig. 3(a) as an example, all the transistors' size and potentials of gate, drain, source, and body are known, except the four nets' potential marked by the colored circles. To derive the values of unknown potentials, we build a polynomial equation system according to the relationships of I_{ds} among transistors, which treats these unknown potentials as variables. After solving the polynomial equation system, the values of those unknown potentials are obtained, then the values of all the small-signal model parameters can be derived by the transistor model built before.

The last step is symbolic analysis, as depicted by the mapping ③ in Fig. 4. In this work, we have selected GPDD [9] as our symbolic analysis engine due to its efficiency and reliability. After building a GPDD structure according to the small-signal model of the circuit, the values of dc gain (AC) and phase margin (PM) can be directly calculated through the GPDD structure [9]. If the results satisfy both the specifications of dc gain and phase margin, the state will be passed to the simulator to be further evaluated; otherwise, a reward that is equal to zero is returned.

IV. Experimental Results

The proposed sizing method was mostly implemented in python, with the curve-fitting done by Matlab, and the simulation carried out by Cadence SPICE simulation tool. The experiment was based on a CMOS 65nm process, which can be readily replaced by other CMOS technologies. We now report the experimental result through an OpAmp circuit, the folded-cascode OpAmp depicted in Fig. 5.

A. Sizing Result

As given in Table I, in our experiment, each episode contains 50 steps, which would produce 50 tuples (states,

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 18:05:54 UTC from IEEE Xplore. Restrictions apply.

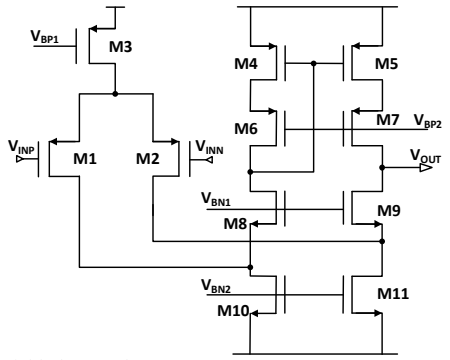


Fig. 5. The folded-cascode OpAmp.

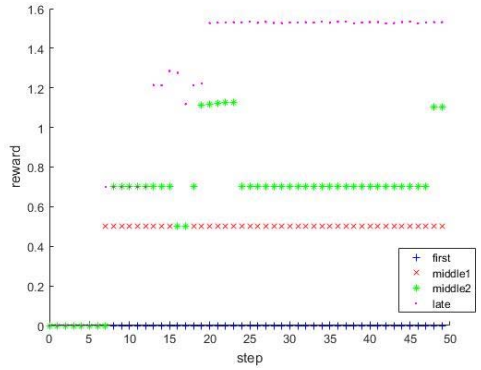


Fig. 6. Rewards obtained at different periods.

actions, and rewards) to the PGNN as the mini-batch training data. The PGNN utilized the artificial neural network (ANN) algorithm, which contains three hidden layers and one softmax layer. Only four performance attributes are considered in the experiment. Their specifications are given at the top three rows in Table II. The weights that assigned to these performance attributes in the reward function (4) are 0.3, 0.2, 0.3, and 0.2 for dc gain, unit gain bandwidth, phase margin and gain margin, respectively. The sum of these weights is equal to 1.

From Table II, one can see that the initial biases and sizes cannot even make all transistors into conduction. After applied our proposed automated sizing method, solutions with good performance were derived. One solution is given in Table II.

B. Learning Process

Fig. 6 briefly illustrates the learning process of the sizing work. As can be seen, in the early period, all trials got zero rewards since the size values did not pass the symbolic filter. This period lasted quite long, and the symbolic filter played an important role in this period, thus greatly improved efficiency. Compared with the early period, the trials in the first middle period quickly passed the symbolic filter and got non-zero rewards by simulation. It indicates that the agent got experience from previous learning and can quickly go to the trained level. In the second middle period, some rewards obtained are larger than 1, which means some feasible solutions had appeared. In the late period, trials got rewards larger than 1.5, which indicates its performance was largely improved compared with the target specification.

Table I. Hyperparameters used in the experiment

#Episode	*#Step*	*Learning Rate*	*Discount Factor*	*Change Rate*
1000	50	0.1	0.95	0.05

Table II. Sizing result of folded-cascode OpAmp

Performance Specification						
DC Gain	60 dB		*Unit Gain Bandwidth*		10 MHz	
Phase Margin	60 °		*Gain Margin*		40 dB	
Folded-cascode OpAmp						
Sizing Result					**Performance Result**	
	Initial values		***Improved values***			***Initial***	***Improved***
VINP	0.4v		0.645v		*DC Gain*	-123.7 dB	61.5 dB
VBP1	0.4v		0.89v				
VBP2	0.4v		0.715v				
VBN1	0.4v		0.505v		*Unit Gain Bandwidth*	X	21.29 MHz
VBN2	0.4v		0.295v				
	Length	*Width*	*Length*	*Width*			
M1, M2	400n	10µ	400n	28µ	*Phase Margin*	X	80.32 °
M3	400n	10µ	400n	30µ			
M4, M5	400n	10µ	600n	6µ			
M6, M7	400n	10µ	600n	6µ	*Gain Margin*	X	51.5 dB
M8, M9	400n	10µ	600n	5µ			
M10, M11	400n	10µ	600n	18µ			

V. CONCLUSION

In this work, we have proposed a self-improvement approach that combines reinforcement learning with deep learning for automated analog circuit sizing. Through observing the sizing process, we found that the algorithm continuously learned from trials and improved itself to find the solutions. In order to greatly improve the efficiency, we have also proposed a fast evaluation method that fits our scenario, but it can be widely applied to fast evaluate circuit performance for other works.

ACKNOWLEDGMENT

This work was supported in part by Natural Sciences and Engineering Research Council of Canada (NSERC), Canada Foundation for Innovation (CFI), Research and Development Corporation (RDC) of Newfoundland and Labrador (through its Industrial Research and Innovation Fund and ArcticTECH R&D Award), and Memorial University of Newfoundland.

REFERENCES

[1] R. A. Rutenbar, G. G. E. Gielen, and J. Roychowdhury, "Hierarchical modeling, optimization, and synthesis for system-level analog and RF designs," *Proc. IEEE*, vol. 95, no. 3, pp. 640–669, 2007.

[2] L. Zhang and U. Kleine, "A novel analog layout synthesis tool," *Proc. IEEE International Symposium on Circuits and Systems*, Vancouver, pp. V101-V104, 2004.

[3] S.E. Sorkhabi and L. Zhang, "Automated topology synthesis of analog and RF integrated circuits: A survey," *Integration, the VLSI Journal,* vol. 56, pp.128-138, 2017.

[4] T. Liao and L. Zhang, "Parasitic-aware GP-based many-objective sizing methodology for analog and RF integrated circuits," *Proc. Asia and South Pacific Design Automation Conference*, Chiba, pp. 475-480, 2017.

[5] A. A. I. Ahmed and L. Zhang, "Fast parasitic-aware synthesis methodology for high-performance analog circuits," *Proc. IEEE*

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 18:05:54 UTC from IEEE Xplore. Restrictions apply.

International Symposium on Circuits and Systems, Seoul, pp. 2155-2158, 2012.

[6] W. Lyu, P. Xue, et al., "An efficient bayesian optimization approach for automated optimization of analog circuits," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 65, no. 6, pp. 1954-1967, 2018.

[7] R. S. Sutton and A. G. Barto, "Reinforcement learning: an introduction," *Cambridge: MIT Press*, 2011.

[8] P. Rajeswari, G. Shekar, et al. "Geometric programming-based power optimization and design automation for a digitally controlled pulse width modulator." *Circuits, Systems, and Signal Processing*, vol 37, no. 9, pp. 4049-4064, 2018.

[9] Z. Zhao, T. Liao and L. Zhang, "Fast performance evaluation for analog circuit synthesis frameworks," *Proc. IEEE International Symposium on Circuits and Systems*, Florence, pp. 1-5, 2018.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 18:05:54 UTC from IEEE Xplore. Restrictions apply.